

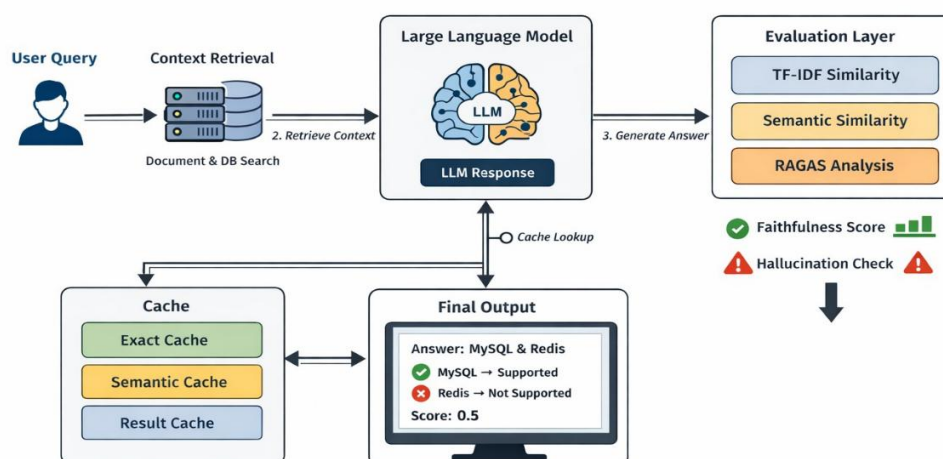
# AI Knowledge Source Detection: Research Document

## Executive Summary

AI Knowledge Source Detection is a critical capability for ensuring the reliability of Large Language Model (LLM) outputs in knowledge-grounded applications. This research document provides a comprehensive analysis of detection methods, tools, benchmarks, and implementation strategies. Key findings:

- **Primary Challenge:** Distinguishing between model-generated knowledge (from training data) vs. context-provided knowledge
- **Leading Approaches:** TF-IDF similarity, semantic embeddings, NLI-based entailment, and LLM-as-judge frameworks
- **Recommended Stack:** TF-IDF + RAGAS Faithfulness for production systems
- **Open Challenge:** No universal metric works across all domains

LLM Hallucination Detection System Architecture



# **Table of Contents**

- 1. Introduction to Knowledge Source Detection**
- 2. Core Detection Methodologies**
- 3. Advanced Research Techniques**
- 4. Production Tools & Frameworks**
- 5. Implementation Architecture**
- 6. Benchmark Comparison**
- 7. Case Studies & Real-World Impact**
- 8. Recommendations & Next Steps**
- 9. References**

# 1. Introduction

## 1.1 Problem Definition

Knowledge Source Detection determines whether an LLM response is:

- **Grounded** in provided context (retrieved documents, knowledge base)
- **Extrinsic** (from training data, potentially outdated)
- **Fabricated** (hallucinated, not supported by any source)

### Example Scenario:

Context: "Dialog router supports 150 Mbps maximum throughput"

Query: "What router should I buy?"

Bad Response: "Buy Starlink - supports 200 Mbps globally" ❌

Good Response: "Dialog router supports up to 150 Mbps" ✅

## 1.2 Why It Matters

- **Legal Liability:** Courts sanction fabricated citations (Mata v. Avianca, 2023)
- **Patient Safety:** Medical advice must reference current guidelines
- **Customer Trust:** Wrong product specs damage brand reputation
- **Regulatory Compliance:** EU AI Act requires transparency in sourcing

## 2. Core Detection Methodologies

### 2.1 Lexical & Statistical Methods

- TF-IDF + Cosine Similarity (Recommended Starter)

#### How it works:

1. Vectorize query and context using TF-IDF
2. Compute cosine similarity: **score = cos(q\_vector, context\_vector)**
3. Threshold: **< 0.3** → Likely out-of-scope → Flag for review

#### Python Implementation:

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
from sklearn.metrics.pairwise import cosine_similarity
```

```
def detect_source_tfidf(query, context, threshold=0.3):
```

```
    vectorizer = TfidfVectorizer()
```

```
    vectors = vectorizer.fit_transform([query] + context)
```

```
    similarity = cosine_similarity(vectors[0:1], vectors[1:])[0]
```

```
    max_sim = max(similarity)
```

```
    return "Grounded" if max_sim > threshold else "Extrinsic"
```

**Strengths:** Zero-cost, fast, no external APIs **Weaknesses:** Misses semantic equivalence (car ≠ automobile)

## 2.2 Semantic Embedding Methods

1. BERTScore • Sentence-BERT • OpenAI Embeddings

### Process:

Query → Embedding → Cosine Similarity → Context Embeddings → Score

**Performance:** 15-25% better than TF-IDF on paraphrased queries

## 2.3 NLI-Based Entailment Checking

Premise: Context | Hypothesis: Generated Answer → {Entailment/Contradiction/Neutral}

### Top Models:

| Model            | Size | F1 Score | Cost |
|------------------|------|----------|------|
| DeBERTa-v3-large | 435M | 0.78     | Free |
| Vectara HHEM-2.1 | 220M | 0.82     | Free |
| T5-Small-NLI     | 60M  | 0.71     | Free |

### 3. Advanced Research Techniques

#### 3.1 Semantic Entropy (Nature 2024) SOTA

1. Generate 8 responses → Cluster by semantic meaning → Compute cluster entropy
2. High entropy = High hallucination risk

Key Insight: Clusters "Paris", "It is Paris", "France's capital is Paris" as identical

#### 3.2 SelfCheckGPT (EMNLP 2023)

1. Same query × 5 → Measure response consistency → Low variance = Likely factual

#### 3.3 LLM-Check (NeurIPS 2024)

1. **45x faster** than multi-generation methods using attention kernel eigenvalues

### 4. Production Tools & Frameworks

#### 4.1 RAGAS (Recommended)

1. Open-source | Context-aware | Research-grade
2. Metrics: Faithfulness (0-1), Context Relevance, Answer Relevancy

Integration Example:

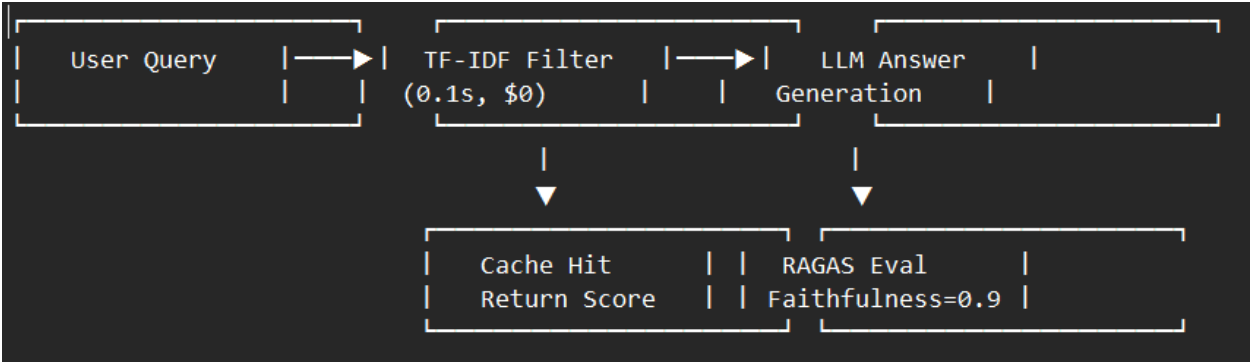
```
from ragas import evaluate
from ragas.metrics import faithfulness

result = evaluate(
    dataset,
    metrics=[faithfulness],
    llm="gpt-4o-mini"
)
# Returns: {"faithfulness": 0.85}
```

4.2 Comparison Matrix

| Tool           | Open Source | Context-Aware | Real-time | Cost |
|----------------|-------------|---------------|-----------|------|
| RAGAS          | ✓           | ✓             | ✓         | Free |
| TruLens        | ✓           | ✓             | ✓         | Free |
| LangSmith      | ✗           | ✓             | ✓         | Paid |
| DeepEval       | ✓           | ✓             | ✓         | Free |
| AWS Guardrails | ✗           | ✓             | ✓         | Paid |

5. Implementation Architecture



Pipeline Components:

- 1. Semantic Cache (FAISS + embeddings)
- 2. TF-IDF Pre-filter (lexical coverage)
- 3. RAGAS Faithfulness (claim-level verification)
- 4. Fallback: NLI Classifier

6. Benchmark Comparison

| Method   Speed   Accuracy   Context Req. |       |      |     |     |        |
|------------------------------------------|-------|------|-----|-----|--------|
| Ground Truth                             |       | Cost |     |     |        |
| TF-IDF                                   | 10ms  | 0.65 | Yes | No  | \$0    |
| BERTScore                                | 50ms  | 0.72 | Yes | Yes | \$0    |
| NLI (DeBERTa)                            | 80ms  | 0.78 | Yes | No  | \$0    |
| RAGAS                                    | 500ms | 0.85 | Yes | No  | \$0.01 |
| Semantic Entropy                         | 5s    | 0.89 | No  | No  | \$0.10 |

**Key Takeaway:** RAGAS offers best accuracy/cost ratio for context-grounded systems